后端批量任务阻塞与多 Worker 改造问题汇总

基于现场测试与日志诊断结果整理

一、结论概述

当前“什么都测试不成功、上传什么都解析半天”的主要原因，不是单个文件本身异常，也不是简单的容器 CPU 配额不足，而是后端采用 Uvicorn 单进程单 worker，长时间批量任务与在线轻量请求共用同一个服务进程，导致其他请求排队、超时甚至 healthcheck 失败。

同时，系统又不能直接把 Uvicorn 改成多 worker。因为当前仍存在进程内状态：**parse_store** 使用 Python 内存字典保存解析结果，取消机制使用 **threading.Event()**。这些状态在不同 worker 之间不共享，直接启用 --workers 2 会引入上传/提交失败、取消失效等新问题。

二、现场现象与实测证据

- 轻量 GET 请求 dictionaries/history 在批量任务运行期间，30 秒无响应并超时。
- 一个仅 322 字符的小 txt 文件，files/parse 首次解析耗时 76.6 秒，正常情况下应小于 1 秒。
- 同一文件在批量任务结束后的再次解析，仅耗时 4.75 毫秒。
- 第一次慢测期间，另一个用户的批量 GLM 任务实际上仍在持续运行，直到约 19:21 才真正结束。
- 当 GLM 任务停止后，服务恢复正常；一旦再次启动批量任务，其他用户的请求仍可能重新卡住。

时机	files/parse 耗时	轻量 GET
有人在跑批量 GLM 任务	76.6 秒	30 秒超时无响应
批量任务结束后的空闲状态	4.75 毫秒	正常

三、当前问题汇总

优先级	问题	表现	根因/风险
P0	单 worker 导致整个服务被长任务拖住	轻量 GET 30 秒超时，小文件解析 76.6 秒	Uvicorn 当前单进程单 worker；批量任务和普通请求竞争同一进程
P0	CPU 密集型任务影响在线响应	PDF 解析、正则、向量化期间轻量请求无法及时处理	CPU 密集计算与 GIL/线程竞争使事件循环和请求处理能力下降
P0	不能直接改成多 worker	加 --workers 2 后上传流程可能随机失败	parse_store 为进程内存字典，不同 worker 不共享

P0	取消机制不支持多 worker	取消请求落到不同 worker 时可能无效	threading.Event() 属于进程内状态
P1	多 worker 后并发参数会成倍放大	任务槽、GLM 并发数随 worker 数翻倍	ASYNC_WORKERS 与 GLM_MAX_CONCURRENCY 为进程级配置
P1	中文功能收到英文输入	日志出现多次“语言不匹配”	属于输入/产品交互问题，不是后端性能 bug

四、核心根因链路

```
用户 A 启动大型批量任务
        ↓
单 Uvicorn worker
        ↓
大量 GLM 调用 + PDF 解析 + 正则 + 向量化 + CPU/GIL 竞争
        ↓
唯一 backend worker 被大量占用
        ↓
用户 B 的 /files/parse、/dictionaries/history、healthcheck 等请求排队
        ↓
GET 30 秒超时、322 字符 txt 解析 76.6 秒、healthcheck 间歇失败
        ↓
用户看到“什么都测不成功”“上传什么都解析半天”
```

关键证据：同一个 322 字符小文件在批量任务期间耗时 76.6 秒，而任务结束后只需 4.75 毫秒。这说明文件本身并不是主要原因，真正问题在于服务进程被其他长任务占用。

五、为什么不能直接改成 --workers 2

1. parse_store 是纯进程内存

```
infrastructure/document_parser/parse_store.py
_STORE: Dict[str, Dict[str, Any]] = {}
```

当前流程：POST /files/parse 将解析结果写入 _STORE，并返回 parse_id；后续请求再携带 preparsed-parse_id，通过 parse_store.get() 取回解析内容。

如果启用多个 worker，第一次请求可能落到 Worker A，而第二次提交请求可能落到 Worker B。由于两个进程内存不共享，Worker B 中查不到 Worker A 保存的 parse_id，最终导致文件“消失”或提交失败。

2. 取消机制也是进程内状态

tool_integration_service.py 使用 threading.Event() 实现取消。多 worker 下，如果任务运行在 Worker A，而取消请求被 Worker B 接收，Worker B 设置的 Event 无法影响 Worker A 中正在执行的任务，因此取消/重跑机制存在失效风险。

3. 并发参数会按 worker 数放大

```
当前示例：
ASYNC_WORKERS = 4
GLM_MAX_CONCURRENCY = 6

若 Uvicorn workers = 2，则全局可能变为：
任务槽约 4 × 2 = 8
GLM 并发约 6 × 2 = 12
```

因此，多 worker 改造还必须同时重新评估 GLM 侧 QPS、CPU、线程数和内存占用，不能只增加一行启动参数。

六、目前已确认可支持多 worker 的部分

任务进度与结果查询本身已经走 MySQL 持久化，包括 analysis_tasks、task_items、result_records 等表。GET /tasks/{id}/progress 和结果查询可由任意 worker 读取，因此这一部分具备跨 worker 可见性。

七、当前状态判断

- 当批量 GLM 任务停止时，服务可以恢复流畅，说明系统并非持续性不可用。
- 只要再次有人启动长时间批量任务，其他用户的上传、查询、词典加载和健康检查仍可能被拖慢或超时。
- “语言不匹配”属于独立问题：功能面向中文文献，但有人提交了疑似英文文本；该报错本身合理，但前端应更明确提示输入语言要求。

八、建议整改顺序

第一步：将 parse_store 从进程内存迁移到 Redis/MySQL，或调整流程，使后续提交不依赖某个 worker 的本地 parse_id。

第二步：将取消/重跑状态从 threading.Event() 改造成可跨进程共享的状态，例如 Redis/MySQL 中的任务取消标记。

第三步：重新核算 ASYNC_WORKERS、GLM_MAX_CONCURRENCY 等进程级参数，避免启用多个 Uvicorn worker 后全局并发翻倍。

第四步：完成上述改造后，再启用 Uvicorn 多 worker，例如 --workers 2，并进行双用户并发压力测试。

第五步：补充前端输入约束与错误提示，明确中文工具不应提交英文文本，减少“测试失败”误判。

九、最终判断

当前架构的核心矛盾是：单 worker 时进程内状态正常，但长任务会拖住整个在线服务；直接多 worker 又会使 parse_store 和取消机制失效。

因此，正确方向不是简单重启，也不是直接加 --workers 2，而是先把关键进程内状态改造成跨进程可共享状态，再启用多 worker 隔离长任务与轻量请求，同时重新核算全局并发配置。